Rekursion - Terminierung

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Verzweigte Rekursion
- Wechselseitige Rekursion
- Verschachtelte Rekursion

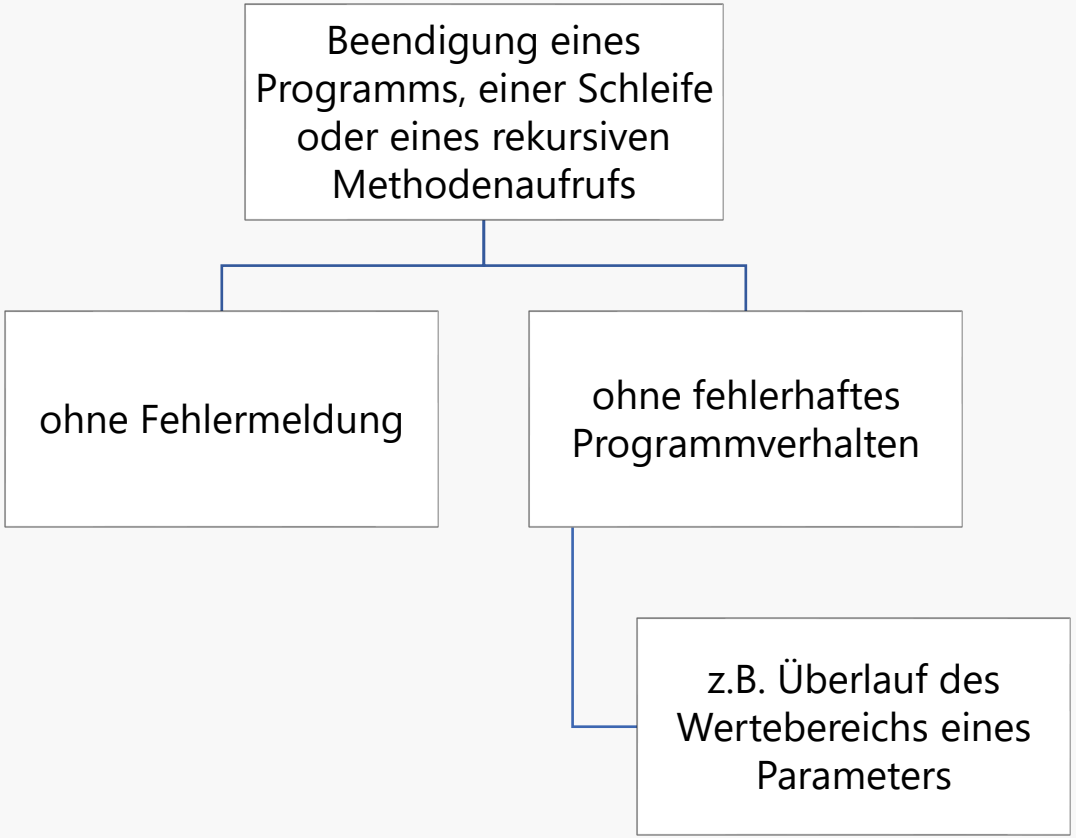
Was wird in diesem Foliensatz besprochen?

- Terminierung
- Muster für Abbruchbedingungen
- Muster für Fortschritt

Was lernen Sie kennen?

- Überlegungen zur Terminierung von rekursiven Methoden
- Muster für Abbruchbedingungen bei rekursiven Methoden
- Muster für Fortschritt bei rekursiven Methoden

Reguläre Terminierung



Beispiele für Methoden, die (meist) nicht regulär terminieren

```
private static void endless() {
    endless();
}

private static void overflow(int x) {
    if (x * x > 0) {
        overflow(x * x);
    }
}
```

Fundiertheit und Fortschritt

Fundiertheit
• Eine rekursive Methode ist **fundiert**, wenn es eine erreichbare Abbruchbedingung gibt

Fortschritt
• Methodenausführung erzielt einen **Fortschritt**, wenn sie uns näher an die Abbruchbedingung bringt

Reguläre Terminierung setzt Fundiertheit und einen Fortschritt um einen Mindestbetrag in jedem Rekursionsschritt voraus

Praktisches Beispiel (Fakultät)

$$n! = \begin{cases} 1 & n = 0 \\ n \cdot (n-1)! & n > 0 \end{cases}$$

```
// n >= 0
private static long factorial(int n) {
    return n <= 0 ? 1 : n * factorial(n - 1);
}
```

Fundiertheit und Fortschritt
• Terminiert theoretisch • Aber praktisch (Java) nur korrekte Werte für 0 <= *n* < 21

Fakultät – redundanter Basisfall

```
private static long factorialRedundant(int n) {
    return n <= 0 || n == 1 ? 1 : n * factorialRedundant(n - 1);
}
```

Terminierung

- Ähnlich zu vorheriger Implementierung

Hinweis

- Ein Basisfall zu viel
  - Kann auch mit ursprünglichem Basisfall berechnet werden
- Redundanter Basisfall bringt nicht viel und verwirrt

Fakultät – falscher Basisfall

```
private static long factorialMissingBaseCase(int n) {
    return n == 1 ? 1 : n * factorialMissingBaseCase(n - 1);
}
```

Terminierung
• Terminiert für $n > 0$ • Funktioniert nicht für $n <= 0$ (theoretisch Endlosrekursion)

Fakultät – kein Basisfall

```
private static long factorialNoBaseCase(int n) {
    return n * factorialNoBaseCase(n - 1);
}
```

Terminierung
• Nicht fundiert • Ruft sich immer wieder auf (theoretisch Endlosrekursion)

Fakultät – falscher rekursiver Aufruf

```
private static long factorialNoConvergence(int n) {
    return n <= 0 ? 1 : factorialNoConvergence(n) * n;
}
```

Terminierung
• Fundiert • Fehlerhafter Fortschritt

Endlosrekursion

Bei einigen bisherigen Beispielen

- Rekursion terminiert nicht
- Methode ruft sich immer wieder auf

Praxis in Java

- Führt zu einem Fehler (**StackOverflowError**)

Muster für Abbruchbedingungen

Ganze Zahl n

- Einzelne Zahl: n <= 1
- Distanz von einer Zahl: Math.abs(n) <= 2

Intervall [start, end]

- Leeres Intervall: start > end
- Einzelne Zahl: start == end (oder sicherheitshalber start >= end)

String str

- Leerer String: str.**isEmpty**() liefert true
- Bestimmter Wert: str.**equals**("Test")
- Bestimmtes Anfangszeichen: str.charAt(0) == 'T'

Muster für Fortschritt von Methode m (für ganze Zahl n)

Einzelschritt Richtung Zielwert

- m(n + 1) bzw. m(n - 1)

Mehrfachschritt Richtung Zielwert

- m(n + k) bzw. m(n - k)
- **k Basisfälle möglich**

Muster für Fortschritt von Methode m (Intervall [start, end])

Einzelschritt von links

- start benutzen
- m(start + 1, end)

start	start + 1	start + 2	start + 3	...	...	..	end
start	**start + 1**	start + 2	start + 3	...	...	..	end
start	start + 1	**start + 2**	start + 3	...	...	..	end
start	start + 1	start + 2	**start + 3**	...	...	..	end
			...				

Einzelschritt von rechts

- end benutzen
- m(start, end - 1)

start	...	...	...	end - 3	end - 2	end- 1	**end**
start	...	...	...	end - 3	end - 2	**end- 1**	end
start	...	...	...	end - 3	**end - 2**	end- 1	end
start	...	...	...	**end - 3**	end - 2	end- 1	end
			...				

Muster für Fortschritt von Methode m (Intervall [start, end])

Mehrfachschritt

- m(start + k + 1, end)
- Beispiel für k = 1
  - start und start+1 benutzen
  - m(start + 2, end)

start	**start + 1**	start + 2	start + 3	…	…	..	end
start	**start + 1**	**start + 2**	**start + 3**	…	…	..	end
			…				

Vom Intervall zum String

- Indexbereich eines Strings ausnutzen
- Typischer Rekursionsfortschritt bei Strings
  - String str = "Test";

0	1	2	3
T	e	s	t

  - Von links str.substring(1)

0	1	2	3
T	e	s	t

  - Von rechts str.substring(0, str.length() - 1)

0	1	2	3
T	e	s	t